1. Cours

- **différence entre famine et interblocage ?** f : activité est indéfiniment retardée car d'autres activités monopolisent les ressources, même si les ressources nécessaires sont disponibles. Cela peut arriver en raison d'une politique d'allocation non équitable. i (deadlock) : situation où un ensemble d'activités se bloque mutuellement car chacune attend une ressource détenue par une autre, sans possibilité de progression.
- **Différence entre exclusion mutuelle et transaction :** em : Utilisée pour empêcher plusieurs activités d'accéder simultanément à une section critique du code. Elle garantit une exécution ordonnée, mais n'offre pas de reprise en cas d'échec. T : Une séquence d'opérations qui s'exécute entièrement (atomicité) ou pas du tout. Elle inclut des mécanismes pour gérer les erreurs ou interruptions (rollback).
- **Contrôle de concurrence : estampilles vs certification : e :** Restreint davantage le parallélisme car les transactions sont ordonnées strictement selon des horodatages fixes. Si une transaction échoue, elle doit être replanifiée. C : Permet davantage de parallélisme en validant les transactions uniquement à leur fin. Elle n'impose pas d'ordre rigide en avance.
- **Cas du contrôle de concurrence basé sur le verrouillage à deux phases :** pas d'effet domino car Dans ce schéma, toutes les ressources nécessaires à une transaction sont acquises avant le début de l'exécution (phase de verrouillage croissant), puis libérées en une seule fois (phase de verrouillage décroissant). L'absence d'effet domino est garantie car une transaction ne peut pas libérer partiellement ses ressources avant la fin de son exécution, empêchant ainsi que d'autres transactions soient affectées par des modifications partielles.

2. Problème

- 1 : proche du problème des lecteurs/rédacteurs. Différences : Le problème ici impose des restrictions spécifiques sur la composition des passagers, ce qui n'est pas présent dans les exemples classiques.
- 2 :
  Variables nécessaires :
  mutex : Exclusion mutuelle pour modifier l'état.
  countCapulet, countMontaigu : Compteurs des passagers de chaque famille.
  barrière : Assurer que 4 passagers embarquent avant le départ.
  semLanceur : Permet de choisir un pilote.
  Code :

```
mutex.up();
if famille == Montaigu then
  countMontaigu++;
else
  countCapulet++;
mutex.down();

barrière.attendre(); // Attendre que 4 passagers soient prêts

mutex.up();
if countCapulet + countMontaigu == 4 then
  lanceur = true;
  countCapulet = 0; countMontaigu = 0;
end if
mutex.down();
if lanceur then
    semLanceur.up(); // Permet au lanceur de démarrer
end if
semLanceur.down();
démarrer();
```

- 3 : Conditions d'acceptation des opérations :
  embarquer(famille) : Nombre total ≤ 4 et Pas de combinaison 1/3 entre les familles.
  piloter(lanceur) : Attendre que tous les passagers soient à bord.
- 4 : Variables d'état :
  countCapulet, countMontaigu : Compteurs par famille.
  passagers : Nombre total de passagers embarqués.
  lanceurDisponible : Booléen indiquant si le pilote est choisi.
- 5 : Invariant : 0 ≤ passagers ≤ 4
  (countCapulet, countMontaigu) ≠ (1, 3) ou (3, 1)
- 6 : Code du moniteur :
  condition plein, pret; // Variables condition

```
function embarquer(famille) : boolean {
  attendre(mutex);
  if famille == Montaigu then
    countMontaigu++;
  else
    countCapulet++;
  end if
  passagers++;
  if passagers == 4 then signaler(plein); end if
  relacher(mutex);
  attendre(pret);
  return lanceurDisponible;
}

function piloter(lanceur) {
  attendre(plein);
  lanceurDisponible = true;
  signaler(pret);
}
```

- **7.**
  Contrôle du nombre total de passagers :
  La variable d'état passagers limite le nombre total à 4. Tant que passagers < 4, l'opération embarquer permet à de nouveaux passagers de monter. Une fois passagers = 4, aucun autre passager ne peut appeler embarquer jusqu'à ce que le bac démarre.
  Vérification des conditions de sécurité :
  Les contraintes sur les compositions interdites (1/3 ou 3/1) sont directement codées dans les conditions d'acceptation de embarquer. Si une tentative de substitution viole ces contraintes, l'appel est bloqué jusqu'à ce que la composition redevienne valide.
  Blocage des passagers supplémentaires :
  Une fois les 4 passagers validés, les autres appels à embarquer sont bloqués (via les variables condition et le mutex) jusqu'à la réinitialisation des compteurs après le départ.
- **8.** La propriété ne serait pas garantie dans ce cas à cause du **manque de contrôle sur l'ordre d'exécution des activités signalées** :
  Problème d'ordre des signaux :
  Avec un moniteur à priorité au signaleur, les activités signalées peuvent se réveiller dans un ordre arbitraire, sans garantie FIFO (premier arrivé, premier servi). Cela pourrait permettre à un passager supplémentaire de se substituer à l'un des 4 premiers passagers validés, invalidant la composition.
  Absence de file spécifique :
  Sans file dédiée, les passagers signalés attendent dans une file commune. Si un passager d'une autre famille (ou non prioritaire) se réveille avant un passager qui devrait monter, cela pourrait conduire à un groupe invalide ou à des violations des contraintes de sécurité.
  Risques d'interférence :
  Les passagers signalés de manière concurrente pourraient interférer avec les compteurs d'état (countCapulet, countMontaigu, etc.) et provoquer des incohérences. Par exemple, deux passagers pourraient s'ajouter au même moment, conduisant à un dépassement du nombre maximal autorisé.
- **9 : Automate des états et transitions légales :**
  États possibles :
  Initial : Aucun passager embarqué.
  En cours d'embarquement : Des passagers montent à bord, jusqu'à un total de 4.
  Prêt pour le départ : 4 passagers à bord, avec un pilote désigné.
  Transition en attente : Bac prêt pour un nouveau voyage après un départ.

Transitions légales :
C (embarquerCapulet) : Un Capulet monte à bord, état reste "en cours".
M (embarquerMontaigu) : Un Montaigu monte à bord, état reste "en cours".
D (autoriserDemarrage) : Le bac démarre après désignation du pilote, transition vers "en cours de traversée".
Conditions sur les transitions :
La somme des passagers Montaigu et Capulet doit être ≤ 4.
Pas de situation 1/3 ou 3/1 (sécurité).

- **10 : Code de la tâche Bac :** Voici un extrait, couvrant l'état initial et deux états successifs :

```
task body Bac is
   countCapulet, countMontaigu : Integer := 0;
   totalPassagers : Integer := 0;
   lanceurChoisi : Boolean := False;
begin
   loop
      -- État initial : attendre des passagers
      select
         when totalPassagers < 4 and countCapulet + 1 <= 4 and countMontaigu <= 3 =>
            accept embarquerCapulet do
               countCapulet := countCapulet + 1;
               totalPassagers := totalPassagers + 1;
            end;

         when totalPassagers < 4 and countMontaigu + 1 <= 4 and countCapulet <= 3 =>
            accept embarquerMontaigu do
               countMontaigu := countMontaigu + 1;
               totalPassagers := totalPassagers + 1;
            end;

      or
         -- État prêt à démarrer
         when totalPassagers = 4 =>
            accept autoriserDemarrage(lanceur : out Boolean) do
               lanceurChoisi := True;
               lanceur := lanceurChoisi;
               -- Réinitialisation après le départ
               countCapulet := 0;
               countMontaigu := 0;
               totalPassagers := 0;
            end;
      end select;
   end loop;
end Bac;
```

Transition où le **lanceur = true** : Lors de l'appel à autoriserDemarrage, après que 4 passagers sont embarqués.

- **11 : Automate des états et transitions légales :**

États possibles :
Initial : Aucun passager n'est présent.
En cours d'embarquement : Les passagers Montaigu et Capulet montent.
Prêt à démarrer : 4 passagers sont à bord avec un pilote désigné.
Transitions légales :
canal capulet : Un passager Capulet est ajouté.
canal montaigu : Un passager Montaigu est ajouté.
canal pilotage : Indique au pilote qu'il peut démarrer.
Règles de transition :
Total des passagers ≤ 4.
Pas de combinaison interdite (1/3 ou 3/1).

- **12 : Code de l'activité de synchronisation :**

```
func Bac() {
   capulet := make(chan bool)
   montaigu := make(chan bool)
   pilotage := make(chan bool)

   go func() {
      countCapulet := 0
      countMontaigu := 0
      totalPassagers := 0

      for {
         select {
         // État initial : attendre les passagers
         case <-capulet:
            if totalPassagers < 4 && countCapulet+1 <= 4 && countMontaigu <= 3 {
               countCapulet++
               totalPassagers++
            }

         case <-montaigu:
            if totalPassagers < 4 && countMontaigu+1 <= 4 && countCapulet <= 3 {
               countMontaigu++
               totalPassagers++
            }

         // État prêt à démarrer
         case pilotage <- (totalPassagers == 4):
            if totalPassagers == 4 {
               // Réinitialisation pour le prochain groupe
               countCapulet = 0
               countMontaigu = 0
               totalPassagers = 0
            }
         }
      }
   }()
}
```

Transition pour le **canal pilotage** : Quand totalPassagers == 4, le pilote est désigné et signalé.

1. **Programmation concurrente**

- 1.a : Problème du code donné :
  Le problème principal est l'interblocage (deadlock). Les activités concurrentes peuvent se bloquer mutuellement si elles acquièrent les verrous dans des ordres différents.
  Exemple :
  Activité 1 verrouille lockx puis attend lockz.
  Activité 3 verrouille lockz puis attend locky.
  Activité 2 verrouille locky puis attend lockx.
  Cela peut créer un cycle d'attente, entraînant un deadlock.
- 1.b : Deux solutions pour corriger ce problème :
  Ordre global de verrouillage : Imposer un ordre fixe pour acquérir les verrous, par exemple : toujours acquérir les verrous dans l'ordre lockx, locky, lockz.
  Utilisation d'un mécanisme sans blocage : Implémenter un mécanisme de détection de deadlock ou utiliser des verrous avec timeout pour éviter un blocage infini.
- 2 : Avantages des pools de threads :
  Réduction des coûts : Limite les coûts liés à la création et destruction de threads.
  Meilleure gestion des ressources : Permet de limiter le nombre maximal de threads, évitant la surcharge du système.
  Performance accrue : Réutilisation des threads existants améliore les performances.
- 3 :Recherche du maximum dans un tableau :
  Utiliser un schéma fork/join est préférable, car :
  Il divise la recherche en tâches parallèles sur des segments du tableau, ce qui est efficace pour des calculs bien structurés.
  Les pools de threads sont plus adaptés pour des tâches récurrentes ou à durée indéterminée.

2. **Moniteur**

- 1 :Conditions d'acceptation des opérations :
  Un client peut s'asseoir si le siège est libre et si l'apprenti ne passe pas le balai.
  Le barbier commence le rasage si un client est assis.
  Le barbier termine le rasage et libère le siège.
  L'apprenti peut balayer après 3 clients rasés, si le siège est libre.
  Aucun client ne peut s'asseoir pendant le balayage.
- 2 : Variables d'état :
  nbClientsRases : Nombre de clients rasés.
  siegeOccupe : Booléen indiquant si le siège est occupé.
  balayageEnCours : Booléen indiquant si l'apprenti balaie.
- 3 : Invariant :
  À tout moment, le siège est occupé par un seul client, ou libre.
- 4 : Variables condition :
  condClient: Signale qu'un client peut s'asseoir.
  condBarbier: Signale que le barbier peut raser.
  condApprenti: Signale que l'apprenti peut balayer.
- 5 :Code du moniteur

```
class Moniteur {
  int nbClientsRases = 0;
  boolean siegeOccupe = false;
  boolean balayageEnCours = false;

  Condition condClient, condBarbier, condApprenti;

  void clientSassoir() {
    attendre(condClient, !siegeOccupe && !balayageEnCours);
    siegeOccupe = true;
    signaler(condBarbier);
  }

  void barbierRaser() {
    attendre(condBarbier, siegeOccupe);
    // Rasage en cours
    siegeOccupe = false;
    nbClientsRases++;
    if (nbClientsRases % 3 == 0) {
      balayageEnCours = true;
      signaler(condApprenti);
    } else {
      signaler(condClient);
    }
  }

  void apprentiBalayer() {
    attendre(condApprenti, balayageEnCours);
    // Balayage en cours
    balayageEnCours = false;
    signaler(condClient);
  }
}
```

- Modifications pour l'extension :
  Ajouter une variable clientsEnAttente pour suivre les clients en file d'attente.
  L'apprenti aide si clientsEnAttente > 5.
- 6. Modifications du code de l'apprenti pour l'extension
  Lorsque l'apprenti devient autonome et aide le barbier :
  L'apprenti peut effectuer des rasages si le nombre de clients en attente dépasse 5.
  Ajout d'une condition pour distinguer le mode "balayage" et le mode "rasage".
  Pseudo-code pour l'apprenti (mode autonome) :

```
void apprenti() {
  while (true) {
    attendre(condApprenti, balayageEnCours || clientsEnAttente > 5);
    if (balayageEnCours) {
      // Balayer
      balayageEnCours = false;
      signaler(condClient);
    } else if (clientsEnAttente > 5) {
      // Raser un client
      clientsEnAttente--;
      // Rasage en cours
      signaler(condClient); // Libère un client pour être rasé
    }
  }
}
```

  Introduire une nouvelle condition condApprentiRase pour signaler l'apprenti.
- 7. Nouvelles variables d'état
  clientsEnAttente: Nombre de clients en attente dans la file.
  siegeApprentiOccupe: Booléen indiquant si l'apprenti utilise le second siège pour raser.
  siegePrincipalOccupe: Booléen pour indiquer si le siège du barbier est occupé.
- 8. Nouvelles variables condition
  condApprentiRase: Signale à l'apprenti qu'il peut raser un client.
  condFileAttente: Contrôle l'accès à la file d'attente (éviter les conflits entre barbier et apprenti).

3. **Processus communicants / tâche Ada (4 points)**

- 1 :Interface du barbier et de la boutique :
  Barbier : procedure raserClient;
  Boutique :

```
        procedure entrerClient;
        function quitterBoutique return Boolean;
```

- 2 : Code Client :

```
task Client is
begin
  Boutique.entrerClient;
  attendre(Barbier.raserClient);
  Boutique.quitterBoutique;
end Client;
```

- 3 : Code Barbier :

```
task Barbier is
begin
  loop
    attendre(Client);
    raserClient();
  end loop;
end Barbier;
```

- 4 : Boutique :

```
task Boutique is
  procedure entrerClient is
    -- Insère le client dans la file d'attente
  procedure quitterBoutique is
    -- Retire un client après rasage
end Boutique;
```

4. Transactions

- 1.a : Propagation directe : Chaque écriture est immédiatement visible par les autres transactions. Le graphe inclut des arcs entre transactions partageant des variables mises à jour.
- 1.b : Propagation différée : Les écritures sont visibles uniquement à la validation. Les dépendances sont calculées uniquement sur les lectures effectuées avant les écritures différées.
- 2a. Résultat avec protocole de certification et propagation différée
  Dans la certification avec propagation différée :
  Chaque transaction est exécutée en isolation.
  Lors de la validation, les écritures sont propagées et les conflits détectés.
  Une transaction est annulée si elle est en conflit avec une autre transaction validée.
  Déroulement :
  T1 est validée (lecture de x = 0 et z = 0).
  T2 est validée (écrit y = 2, lit x = 0).
  T3 :Écrit x = 3, z = 1.
  Lit y = 2 écrit par T2, donc conflit. T3 est annulée.
  T4 est validée (lit y = 2 et z = 0, car T3 est annulée).
  Résultat final :
  x = 0, y = 2, z = 0.
  Transactions validées : T1, T2, T4.
  Transactions annulées : T3.
  Ordre série équivalent : T1 → T2 → T4.
- 2b. Résultat avec protocole d'estampilles (timestamps)
  Les estampilles sont attribuées selon l'ordre de lancement des transactions :
  T1 (TS=1), T2 (TS=2), T3 (TS=3), T4 (TS=4).
  Déroulement :
  T1 est validée (aucun conflit avec les autres transactions).
  T2 est validée (lit x = 0 avant l'écriture de T3, donc pas de conflit).
  T3 tente d'écrire x = 3 et z = 1. Cependant, ces écritures se produisent après des lectures (z par T1 et y par T2) incompatibles avec son estampille. T3 est annulée.
  T4 est validée (lit y = 2 écrit par T2, aucun conflit).
  Résultat final :
  x = 0, y = 2, z = 0.
  Transactions validées : T1, T2, T4.
  Transactions annulées : T3.
  Ordre série équivalent : T1 → T2 → T4.
- 2c. Résultat avec verrouillage à deux phases strict
  Dans ce protocole :
  Les verrous sont acquis avant chaque opération et libérés uniquement à la fin.
  Si un verrou est indisponible, la transaction est bloquée.
  Déroulement :
  T1 acquiert les verrous sur x et z pour lire, puis les libère. Validée.
  T2 acquiert un verrou sur y pour écrire et sur x pour lire. Validée.
  T3 acquiert les verrous sur x, z (pour écrire) et sur y (pour lire). Cependant, T2 a déjà écrit y. Conflit : T3 est annulée.
  T4 acquiert les verrous sur y et z pour lire. Validée.
  Résultat final :
  x = 0, y = 2, z = 0.
  Transactions validées : T1, T2, T4.
  Transactions annulées : T3.
  Ordre série équivalent : T1 → T2 → T4.

Systèmes concurrents : TD1

Le problème du barber shop au CBD (93 Seine St-Denis)

1 Le problème du barbier

Il s'agit de modéliser l'activité coordonnée d'un barbier et de ses clients, à l'aide de processus synchronisés.

La boutique du barbier comporte un salon avec un unique fauteuil, et une salle d'attente avec N chaises. Lorsqu'un client se présente, il entre dans la salle d'attente s'il y a des chaises libres, sinon il attend dehors. Chaque fois que le barbier a fini de raser un client, il le lui signale, ce dernier quitte la boutique, puis le client suivant peut entrer dans le salon, pour se faire raser. Les clients entrent dans le salon un par un, lorsque vient leur tour. Lorsque le barbier n'a plus de client à raser, il s'endort. Lorsqu'un client trouve le barbier endormi et que c'est son tour, il le réveille et se fait raser.

Questions à étudier

- Donner une solution au problème avec un moniteur ;
- Considérer le cas où un client abandonne si la salle d'attente est pleine ;
- Considérer le cas où il y a plusieurs barbiers et fauteuils dans le salon.

Interface

> Il ya **2** ressources a partager : le fauteuil et la salle d'attente

- Client :
  - Boucle :
    - **Entrer** dans la *salle d'attente*.
    - // **Patienter** dans *la salle d'attente*. (attente passive)
    - **S'assoir** sur le *fauteuil*.
    - // **Patienter** pendant le rasage (attente passive)
    - **Quitter** le salon.
  - Fin Boucle

- Barbier :
  - Boucle :
    - **Commencer** le rasage.
    - **Terminer** le rasage.
  - Fin Boucle

Prédicats d'acceptation

- **Entrer** : Il y a au moins une place de libre dans *salle d'attente*.
- **S'assoir** : Le *fauteuil* est vide.
- **Quitter** : Le client n'a plus de barbe.
- **Commencer** : il y a un client barbu sur le *fauteuil*.
- **Terminer** : / (facultatif l'application dispose en son sein des informations nécéssaires).

Variables d'états

```
FauteuilLibre : bool := true
nbPlaces : int := N
barbePresente : bool := false
```

- **Entrer** : `nbPlacesLibres > 0`
- **S'assoir** : `FauteuilLibre`
- **Quitter** : `!barbePresente`
- **Commencer** : `!FauteilLibre && barbePresente`
- **Terminer** : `null` Pas besoin de finir le travail pour que le barbier arrête

Invariant

```
1 | (0 <= nbPlacesLibres  <= N) && (barbePresente || fauteuilLibre)
```

Variable condition

> A voir comme des fils d'attentes régie par des conditions

```
Salle
Fauteuil
Rasé
Barbe
```

```
//---------------------- Entrer en salle d'attente --------------------

Si !(nbPlacesLibres > 0) alors
    Salle.wait()
FinSi
// ici nbPlacesLibre > 0
nbPlacesLibres--

//Variante2 : Le client abandonne si la salle d'attente est pleine

Si nbPlacesLibre > 0 alors
|    nbPlacesLibres--
|    retourner True
Sinon
|    retourner False
FinSi
//---------------------- S'asseoir sur le fauteuil --------------------

Si !(FauteuilLibre) alors
    Fauteuil.wait()
FinSi
// ici FauteuilLibre = true

FauteuilLibre := False;
BarbePresente := True;
nbPlacesLibre++
barbe.signal() //Lancement de l'action : Commencer le rasage
salle.signal() //Lancement de l'action : Enter en salle d'attente

//---------------------- Se Lever et sortir ------------------------

Si barbePresente alors
    raser.wait()
FinSi
// ici barbePresnete = False
FauteuilLibre := True
Fauteuil.signal() //Lancement de l'action S'asseoir sur le fauteuil

//---------------------- Commencer le rasage ----------------------

Si !(!FauteuilLibre or barbePresente) alors
    barbe.wait()
FinSi
//ici !FauiteuilLibre or barbePresente


//---------------------- Terminer le rasage --------------------
barbePresente := False
raser.signal() //Lancement de l'action Se Lever et sortir

```

Exercice 2 : Lecteur et Rédacteur

Interface

2 interfaces : lecteur et redacteur

Lecteur

- start read
- end read

Rédacteur :

- start write
- end write

Prédicats d'acceptation

- **Start Read** : Aucun rédacteur en écriture et aucun redacteur en attente
- **End Read** : /
- **Start Write** : Aucun Lecteur en lecture et aucun Redacteur en écriture
- **End Write** : /

Variables d'états

```
nbLecteur : int := 0
nbRedac: int := 0
nbRedactEnAttente : int := 0
```

- **Start Read** : nbRedac = 0 et nbRedactEnAttente = 0
- **End Read** : /
- **Start Write** : nbLecteur = 0 et nbRedact = 0
- **End Write** : /

Invariant

```
nbRedac <=1 && (nbRedac == 0 || nbLecteur == 0);
```

Variable condition

```
accesLecture
accesEcriture
```

```
//Les boucles c'est juste un formalisme pour dire que le moniteur fonct

Start Read :
    Debut Boucle
        Si !(nbRed= 0 and nbREdEnAttente = 0) alors
            AccesLec.wait();
        FinSi
        //Ici nbRed = 0 ou nbRedEnAttente = 0
        nbLec++
        AccesLec.signal();
    Fin Boucle

Start Write :
    Debut Boucle
        Si(!(nbRed ==0 and nbLect ==0) alors)
            nbRedAtt++;
            AccesEcriture.wait();
        FSI
            nbRedAtt--:
    Fin Boucle

End Read :
    DébutBoucle
       nbLect--;
       Si nbLect == 0 alors
           //Ici nbLect==0
           AccesRed.signal();
    FinBoucle

End Write:
    Début Boucle
        nbRed--;
        //nbRed == 0 and nbLect == 0
        Si nbReddAtt > 0 alors
            AccesRed.signal()
        Sinon
            AccesLect.signal()
    Fin Boucle
```

Cas d’un fonctionnement FIFO

Prédicat d’acceptation

- **StartRead** : Il n’y a pas de rédaction en cours et je suis en tête de file 'pas d’activité en attente)
- **EndRead** : Rien
- **StartWrite** : Il n’y a pas de redacteur en cours et pas de lecture en cours et pas d’activité en attente
- **EndWrite** : Rien

Variable d’état

```
nbLect : int := 0
nbRed : int := 0
nbActiviteEnAtt : int = 0
```

Variable d’états

```
StartRead :
    nbRed == 0 and nbActEnAttente == 0

StartWrite :
    nbRed == 0 and nbActEnAttente == 0 and nbLect == 0
```

Invariants

```
    ((nbRed == 0 || nbLect == 0) && nbRed =< 1)
```

Variable Condition

```
FIFO
```

```
StartRead:
    Si !(nbRed == 0 && nbActEnAtt == 0) alors
        nbActEnAtt++;
        FiFo.wait();
        nbActEnAtt--;
    FSI
    //Ici (nbRed == 0 && nbActEnAtt == 0)
    nbLect++
    FiFo.signal();

StartWrite:
    Si !(nbRed == 0 && nbActAtt == 0 && nbLect == 0)
        nbActAtt++;
        FiFo.wait()
        nbActAtt--;
    FSI
    // nbRed == 0 && nbActAtt == 0 && nbLect == 0
```

Exercice 3

Protocole

Allouer : nombre de ressources demandés inférieures ou égale au nombre de ressources disponibles
Libérer : rien

Variables d'état

```
int nbDispo := 0
```

Prédicat

ALlouer : nbDispo >= nbDemandé

Invariant

0<=nbDispo<=N

Var conditions

Acces [1 N] (tableau, petit demandeur d'accès à gauche, gors droite)

Allouer(nbDemandé:int)

```
Si(!(nbDemandé<=nbDispo)){
    Acces[nbDemandé].wait()
}
FSI
    {(1) nbDemandé<=nbDispo }
    nbDispo=nbDispo-nbDemande
    {(2) nbDemandé >=0 && nbDispo <=N}
    Reveiller le suivant()
```

Libérer(nbRendu:int)

```
   nbDispo = nbDispo + nbRendu
   Reveiller le suivant()
{(3) nbDispo >=0}
```

Revieller le suivant

```
    int i := 1
    TantQue i<nbDispo && acces[i].vide() Faire
        i++
    FTQ
    Si ¬acces[i].vide() alors
        Acces[i].signal()
    FSI
```

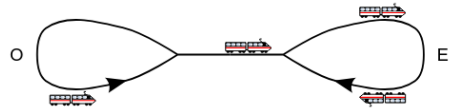
(2)=>(1)
(3) => (1)

Systèmes concurrents : TD3

Thomas the Dank engine

4 Voie unique

Deux villes E et O sont reliées par une ligne de chemin de fer qui comprend un tronçon à voie unique : [1]



Pour s'engager sur le tronçon à voie unique les processus représentant les trains devront faire appel aux actions suivantes, où `direction` appartient à $\{OE, EO\}$:

```
Entrer(direction)
Sortir
```

Les trains font la navette entre E et O. Leur comportement peut donc être simulé par le type de processus suivant :

```
process train is
    maDirection : Direction;
begin
    maDirection ← EO ou OE
    loop
        arriver au tronçon;
        Entrer(maDirection);
        passer le tronçon;
        Sortir;
        arriver à destination;
        maDirection ← inverse(maDirection);
    end loop
end train
```

Questions Proposer une solution basée sur des processus communicants (CSP/Go) dans les cas suivants :

1. un seul train peut se trouver sur le tronçon à voie unique ;
2. un nombre illimité de trains peuvent circuler en même temps sur le tronçon à voie unique (à condition qu'ils aillent dans le même sens) ;
3. au plus N trains peuvent circuler en même temps sur le tronçon à voie unique.

1) Un seul train sur tronçon -> problème exclusion mutuelle
2) on considère un nombre infinie de train sur le tronçon

(http://cacaboudin.fr (http://cacaboudin.fr))

Interface du serveur de synchronisation :

entrer EO
entrer OE
sortir

Code d'un train

> Rq : Le point d'exclamation à la fin du nom des actions signifie que l'ont attends une arrivé d'activité.

[EntrerOE!_ ,sortir!_ ,entrerEO!_ , sortir!]*

condition d'acceptation:

- EntrerOE: pas de trains dans le sens EO
- EntrerEO: pas de trains dans le sens OE
- Sortir: Ø, pas de conditions

Variable d'état:

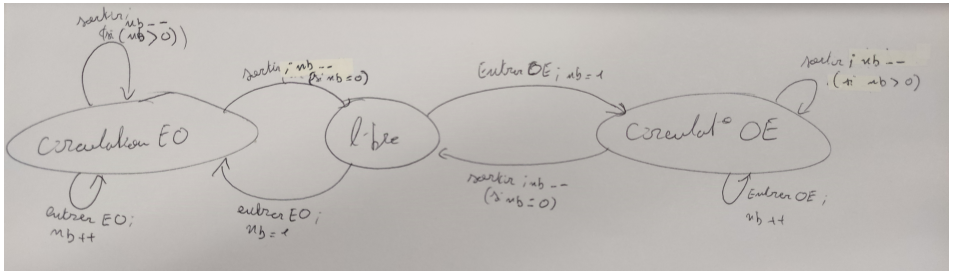
- nbTrainEO : int
- nbTrainEO : int
  OU
- nbTrain: int
- sens : enum (EO,O,E)

Invariant :

- nbtrainOE > 0 $\Longrightarrow$ nbTrainEO == 0 et nbTrainEo > 0 $\Longrightarrow$ nbTrainOE == 0
- nbTrainEO == 0 ou nbTrainEO == 0

> Rq: la deuxieme proposition de variables d'état ne suffit pas à exprimer l'invariant

schéma automate



Code activité Synchronisation

```
Variables
nb : int
Sens : (EO,OE)

Boucle :
Si (Sens == EO ou nb == 0) {
    entrerEO?_; //Attente de l'arrivé d'un train.
    nb++;
    sens = EO;
}
Sinon Si (Sens == OE ou nb == 0){
    entrerOE?_;
    nb++;
    sens = EO;
}
Sinon {
    soritr?_;
    nb--;
}
Fin Boucle
```

```
Variables
etat: (Libre, sensEO, sensOE)
nb: int

boucle:
Si (etat == Libre) {
    [
        entrerEO?_;
        nb = 1;
        etat = sensEO;
    ]

    OU

    [
        entrerOE?_;
        nb = 1;
        etat = sensOE;
    ]

Si(etat = sensEO) {

            [
                entrerEO?_;
                nb++;
            ]

            OU

            [
                sortir?_;
                nb--;
                if(nb==0) {
                    etat = libre;
                }
            ]

        }
}

//La même chose pour sensOE....
```

Tournoi de Coinche

5 Tournoi de bridge

On désire modéliser le déroulement d'un tournoi de bridge (ou de belote). L'organisation du tournoi impose d'assurer que le nombre de personnes présentes dans la salle du tournoi soit toujours un multiple de 4. Il faut donc contrôler les entrées et les sorties selon les contraintes suivantes :

- un joueur peut entrer si un autre est prêt à sortir (échange) ou s'il y a déjà trois autres joueurs en attente d'entrée (le groupe de quatre peut alors entrer) ;
- un joueur peut sortir si un autre est prêt à entrer (échange) ou s'il y a déjà trois autres joueurs en attente de sortie (le groupe de quatre peut alors sortir).

Remarque : on ne s'intéresse pas à l'affectation des joueurs aux tables. Dans le cas de la sortie de quatre joueurs appartenant à des tables différentes, on ne s'intéresse pas à la réorganisation des tables de jeu pour continuer le tournoi.

Question Proposer une solution basée sur des processus communicants (CSP/Go).

Code activité(Joueur)

- DemandeEntrer!_,(DE)
- Entrer!_, (E)
- Jouer,
- demandeSortir!_,(DS)
- ◦ Sortir!_ (S)]*

Condition acceptation

- DE: Rien
- Entrer : Echange (1 DS) ou 3 autres DE
- DS : rien
- S : Echange (1 DE) ou 3 autres DS

Code activité de synchronisation

```
variables
nde, nds, nj, : int;

Boucle :
        { DE?_;
        nde++;
        }
    Sinon {
        DS?_;
        nds++;
    }
    Sinon Si (nde >=1 && nds>=1) {// Echange
        entrer?_;
        sortir?_;
        nde--;
        nds++;
    }
    Sinon Si (nde >= 4) {
        entrer?_;
        entrer?_;
        entrer?_;
        entrer?_;
        nde = nde-4;
    }
    Sinon Si (nds >=4){
        sortie?_;
        sortie?_;
        sortie?_;
        sortie?_;
        nds = nds -4;
    }
    Fin boucle
```